Program 1: Student Grade Calculator Using Switch-Case

Problem Statement

Create a program where users input marks, and the program assigns grades (A, B, C, etc.) using a switch-case statement based on score ranges. Also display “Excellent”, “Average”, or “Needs Improvement” based on grade.

CO & BT Mapping

CO1: Use Java Constructs and apply the paradigms of object oriented programming using Java

BT Level: Apply

Objectives:

• Develop Programming Logic

• Using switch in Java

File Name : Practical.java

```
import java.util.Scanner;

public class Practical {

	public static void main(String args[]) {

		System.out.println("Rohit");

		Scanner sc = new Scanner(System.in);

		System.out.println("Enter your Percentage:");

		int persentage = sc.nextInt();

		sc.close();

		switch(persentage/10) {

		case 9:

		case 10:

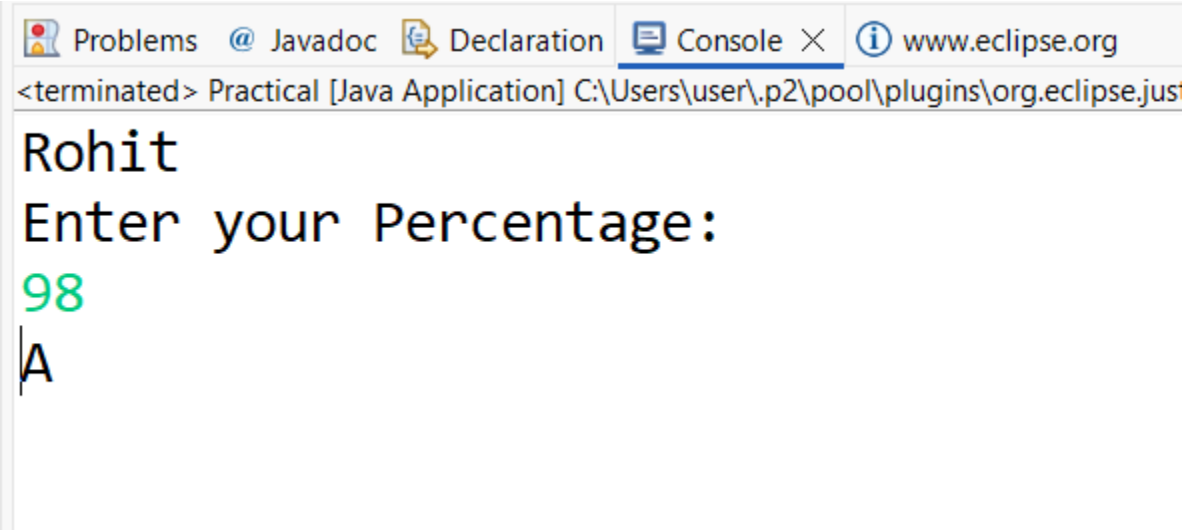
			System.out.println("A");

			break;

		case 8:
```

```
                System.out.println("B");
                break;
            case 7:
                System.out.println("C");
                break;
            case 6:
                System.out.println("D");
                break;
            case 5:
                System.out.println("FF");
                break;
            default:
                System.out.println("FF");
            }
        }
}
```

OUTPUT :



Program 2: Find the LCM of Two Numbers

Problem Statement

Write a program to find the Least Common Multiple (LCM) of two numbers using a loop.

CO & BT Mapping

CO1: Use Java Constructs and apply the paradigms of object oriented programming using Java
BT Level: Analyze and Apply

Objectives:

- Develop Programming Logic
- Using Loops

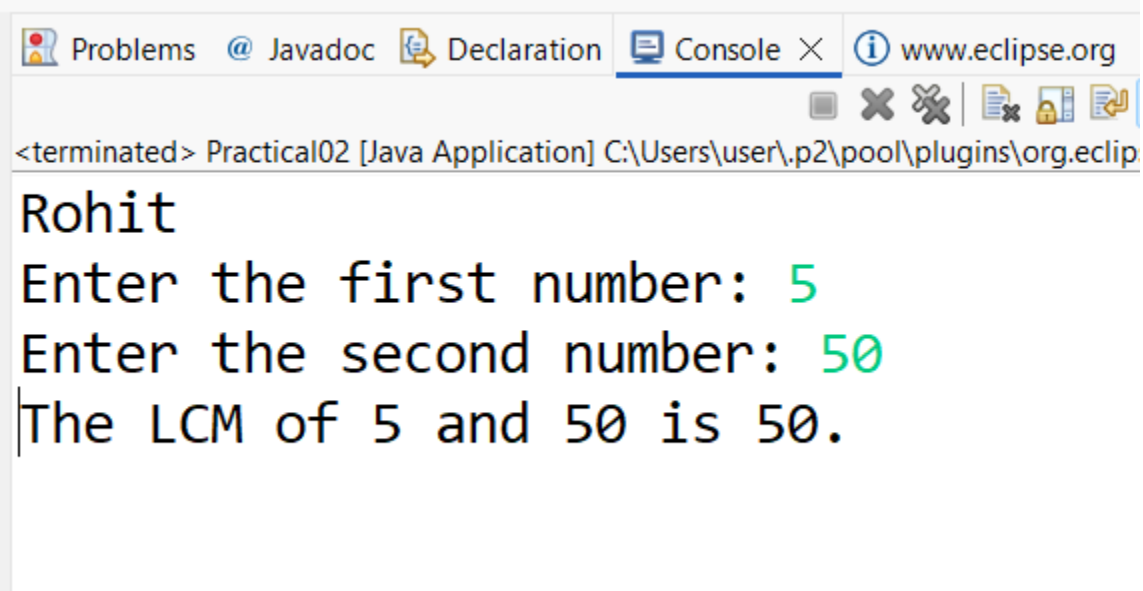
File Name : Practical02.java

```
import java.util.Scanner;
public class Practical02 {
        public static void main(String[] args) {
            System.out.println("Rohit");
            Scanner sc = new Scanner(System.in);
            System.out.print("Enter the first number: ");
            int num1 = sc.nextInt();
            System.out.print("Enter the second number: ");
            int num2 = sc.nextInt();
            sc.close();

            int max = (num1 > num2) ? num1 : num2;
            while (true) {
              if (max % num1 == 0 && max % num2 == 0) {
                 System.out.printf("The LCM of %d and %d is %d.", num1, num2, max);
```

```
                    break;
                }
                max++;
            }
        }
}
```

OUTPUT :



Program 3: Transpose of a Matrix

Problem Statement

Write a program that takes a 2D array (matrix) as input and finds its transpose.

CO & BT Mapping

CO1: Use Java Constructs and apply the paradigms of object oriented programming using Java

BT Level: Apply

Objectives:

- Develop Programming Logic
- Use 2D Arrays in Java

FileName : MatrixTranspose

```
import java.util.Scanner;
public class MatrixTranspose {
	public static void main(String[] args) {
		// TODO Auto-generated method stub
		System.out.println("Rohit");
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter no. of rows:");
			int rows = sc.nextInt();

		System.out.println("Enter no. of columns:");
			int cols = sc.nextInt();
			sc.close();
			int arr [][] = new int [rows][cols];

			int tArr [][]=new int [cols][rows];

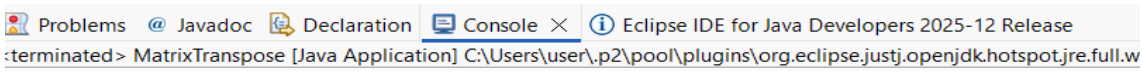
			System.out.println("enter element in array: ");
			for(int i = 0;i<rows;i++) {
				for(int j=0; j<cols;j++) {
					arr[i][j] =sc.nextInt();
				}
	System.out.println();
			}
			System.out.println("Display elements of array arr: ");
			for(int i = 0;i < rows; i++) {
				for(int j=0; j<cols;j++) {
					System.out.print(arr[i][j]);
				}
	System.out.println();
```

```
                    }
                    //Transpose logic
                for (int i = 0; i < rows; i++) {
                   for (int j = 0; j < cols; j++) {
                      tArr[j][i] = arr[i][j];
                   }
                }
                System.out.println("Display Transpose of Matrix:");
            for (int i = 0; i < cols; i++) {
               for (int j = 0; j < rows; j++) {
                  System.out.print(tArr[i][j] + " ");
               }
               System.out.println();
            }

        }

}
```

OUTPUT :




```
Rohit
Enter no. of rows:
2
Enter no. of columns:
5
enter element in array:
5 6 8 4 1

4 5 5 6 8

Display elements of array arr:
56841
45568
Display Transpose of Matrix:
5 4
6 5
8 5
4 6
1 8
```

Program 4: Shopping Cart System with Discounts

Problem Statement

Create a shopping cart system to store item, price, and quantity, then calculate bill. Apply 10% discount if total > 1000.

CO & BT Mapping

CO1: Use Java Constructs and apply the paradigms of object oriented

programming using Java BT Level: Apply

Objectives:

- Creation of Classes and Objects
- Using Inheritance
- Using Array of Objects

File Name : Practical4.java

```
class Product{
    String name;
    double price;
Product(String name, double price){
        this.name=name;
        this.price=price;
    }//constructor
}//class


    class ShoppingCart11{
        Product[]  products;
        int [] qty;
        int cartSize;
```

```
    ShoppingCart11(){
        products =new Product[10];
        qty=new int[10];
    }//

    void add(Product p, int n) {
        products[cartSize]=p;
        qty[cartSize]= n;
        cartSize++;
    }//

void printBill() {
    double total = 0;
        for(int i=0;i<cartSize;i++) {
          double totalItem = products[i].price * qty[i];
          total += totalItem;
          System.out.print("product Name = " + products[i].name+"\n" + "price
= " + products[i].price+"\n" + "quantity = " + qty[i]+"\n");
    }
    if (total > 1000) {
    double discount = total * 0.10;
    total -= discount;
    System.out.print("After Discount Total is"+total);
  }
}//bill
}//class
```

```
public class Practical4 {

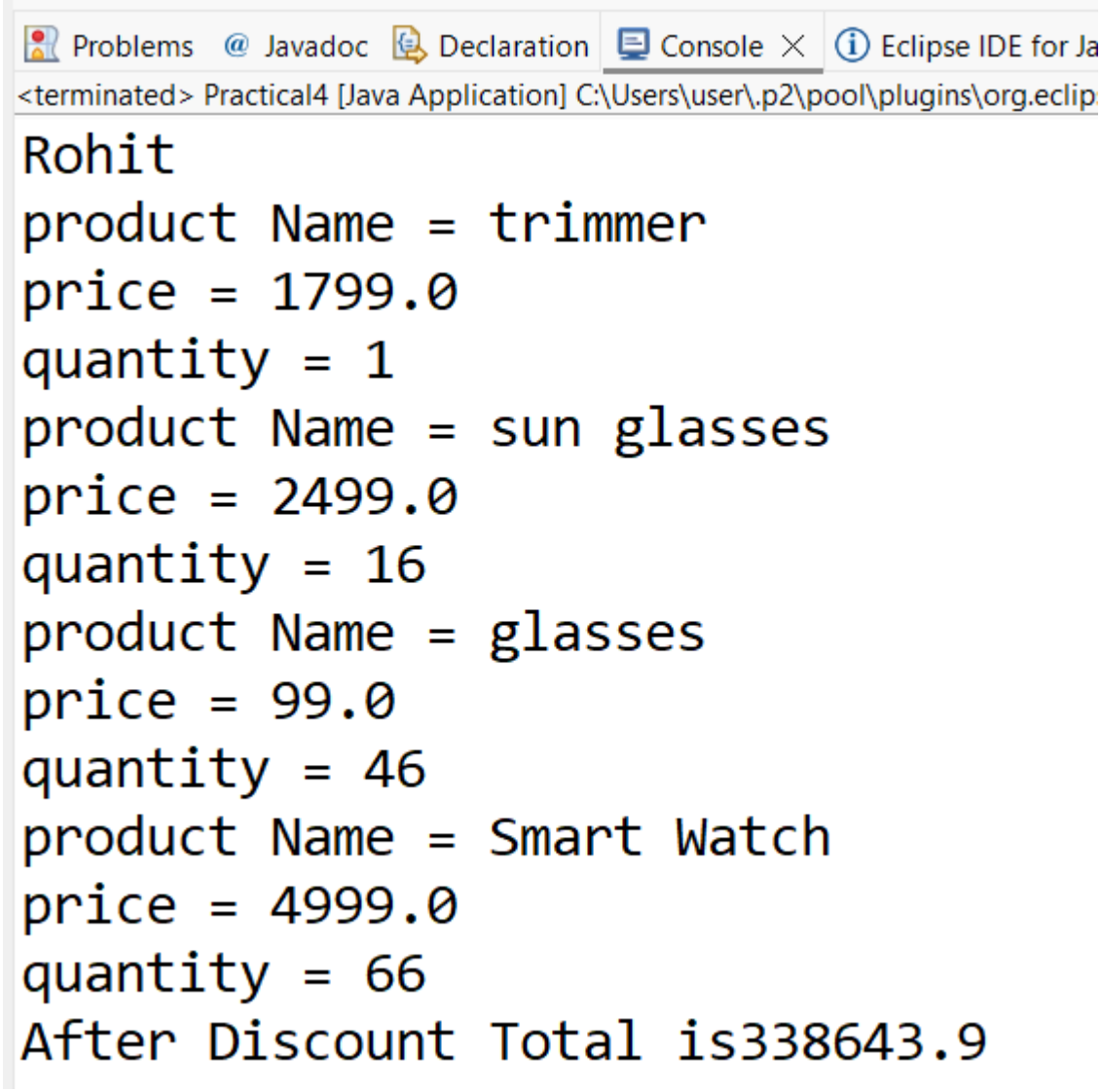

    public static void main(String[] args) {
        System.out.println("Rohit");
        Product p1=new Product("trimmer",1799);
        Product p2=new Product("glasses",99);
        Product p3=new Product("sun glasses",2499);
        Product p4=new Product("Smart Watch",4999);


        ShoppingCart11 s=new ShoppingCart11();


        s.add(p1,1);
        s.add(p3,16);
        s.add(p2,46);
        s.add(p4,66);


        s.printBill();
    }
}
```

OUTPUT:


Problems
Javadoc
Declaration
Console
Eclipse IDE for Ja
<terminated> Practical4 [Java Application] C:\Users\user\.p2\pool\plugins\org.eclip
Rohit
product Name = trimmer
price = 1799.0
quantity = 1
product Name = sun glasses
price = 2499.0
quantity = 16
product Name = glasses
price = 99.0
quantity = 46
product Name = Smart Watch
price = 4999.0
quantity = 66
After Discount Total is338643.9

Program 5: Abstract Car Class with Subclasses

Problem Statement

Create a class Car with abstract methods fuelType() and maxSpeed(). Subclasses DieselCar, PetrolCar, and EV should override these methods.

CO & BT Mapping

CO2: Analyze a given problem and apply concepts of abstract classes, interfaces, dynamic method dispatch and exception handling

BT Level: Apply

Objectives:

- Using Inheritance
- Using abstract classes
- Using super class references

File Name : Vehicle.java

```
abstract public class Vehicle{//partial abst
        String name;
        String type;

        Vehicle(String name, String type){
                this.name=name;
                this.type=type;

        }
        abstract void displayFuelType();

        void displayDetails() {
                System.out.println("Vehicle Name= "+name +" "+"Vehicle type= "+type);
                }
}
```

File Name : Practical4.java

```
class EVCars extends Vehicle{

        String fuel;

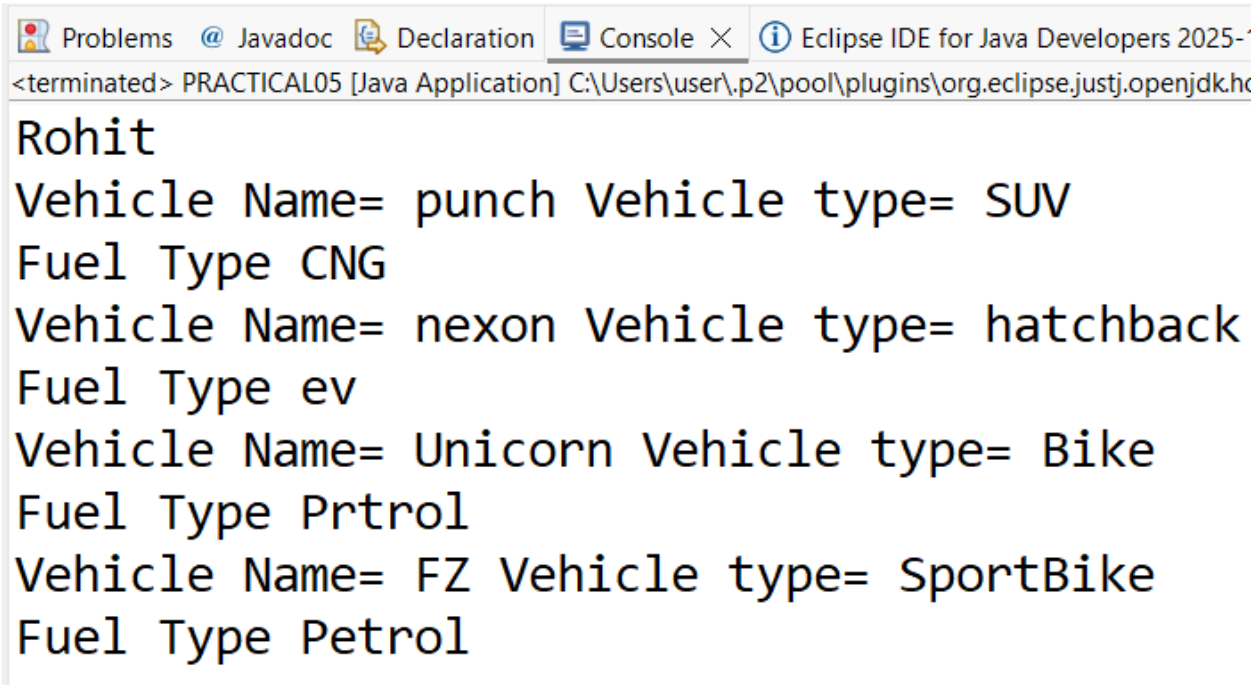
        EVCars(String name, String type , String fuel){

                super (name ,type);
```

```
        this.fuel=fuel;
    }
    void displayFuelType() {
        System.out.println(" "+fuel);
    }
    void displayDetails(){
        super.displayDetails();
        System.out.println("Fuel Type "+fuel);
    }
}
class Petrol extends Vehicle{
    String fuel;
    Petrol(String name, String type , String fuel){
        super (name ,type);
        this.fuel=fuel;
    }
    void displayFuelType() {
        System.out.println(" "+fuel);
    }
    void displayDetails(){
        super.displayDetails();
        System.out.println("Fuel Type "+fuel);
    }
}
public class PRACTICAL05{
```

```
        public static void main(String[] args) {
                System.out.println("Rohit");
                EVCars car=new EVCars("nexon","hatchback","ev");
                Vehicle v1;
                v1=new EVCars("punch","SUV","CNG");
                v1.displayDetails();
                car.displayDetails();
                Petrol car1=new Petrol("FZ","SportBike","Petrol");
                Vehicle v2;
                v2=new EVCars("Unicorn","Bike","Prtrol");
                v2.displayDetails();
                car1.displayDetails();
        }
}
```

OUTPUT :



Program 6: Interface Item – Electronics and Clothing

Problem Statement

Create an interface Item with methods getPrice() and getDescription(). Implement the interface in classes Electronics and Clothing, then create a shopping cart that calculates the total price.

CO & BT Mapping

CO2: Analyze a given problem and apply concepts of abstract classes, interfaces, dynamic method dispatch and exception handling

BT Level: Apply

Objectives:

- Using interfaces
- Using Inheritance
- Using dynamic method dispatch
- Applicaton of concepts to a given problem statement

File Name: Practical06.java

```
package practicalsjava;

interface Item {

double getPrice();

String getDiscription();

}

class Electronics implements Item{

        double price;

        Electronics(double price){

                this.price=price;

        }

        public double getPrice() {
```

```
            return price;
        }
        public String getDiscription() {
            return "Electronic item";
        }
}
class Clothing implements Item{
        double price;
        Clothing(double price){
            this.price=price;
        }
        public double getPrice() {
            return price;
        }
        public String getDiscription() {
            return "Clothing item";
        }
}
class ShoppingCart {
        Item[] products;
        int[] qty;
        int cartSize;
        ShoppingCart() {
        products = new Item[20];
        qty = new int[20];
```

```
        cartSize = 0;

        }
void add(Item i, int n) {

        products[cartSize]=i;

        qty[cartSize]= n;

        cartSize++;

        }
double printBill() {

        double total = 0;

        for(int i=0;i<cartSize;i++) {

         double totalItem = products[i].getPrice() * qty[i];

        total += totalItem;

        System.out.print("product discription =" + products[i].getDiscription() + "\n"+"price =" +
        products[i].getPrice() +"\n"+ "quantity =" + qty[i]+"\n");

        }


        if (total > 1000) {

                double discount = total * 0.10;

                 total -= discount;

                 System.out.print("After Discount Total is"+total);

        }

        return total;

}//bill

}
```

```
public class Practical06 {

        public static void main(String[] args) {

        System.out.println("Rohit");

        ShoppingCart cart = new ShoppingCart();

        Electronics laptop = new Electronics(50000);

        Clothing shirt = new Clothing(1200);

        cart.add(laptop, 1);

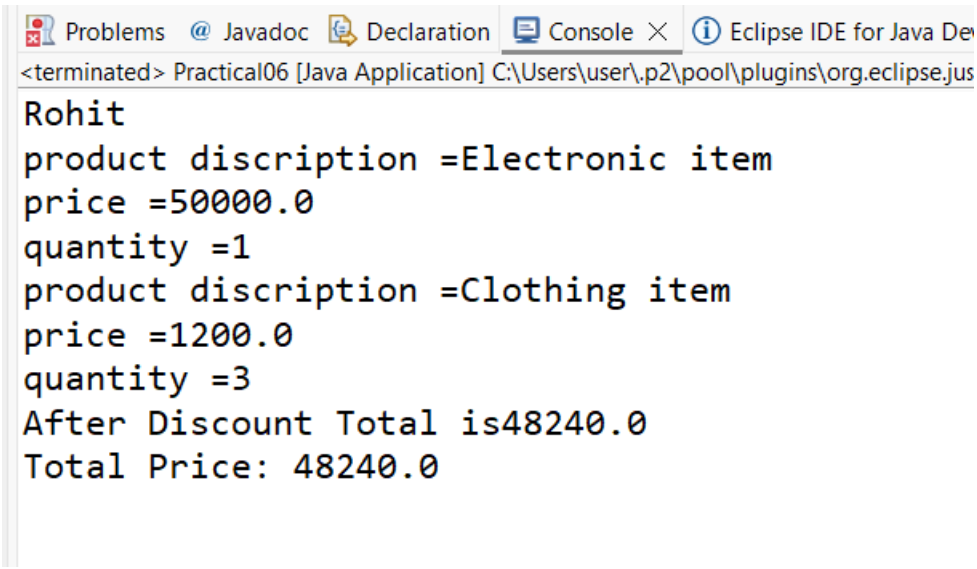
        cart.add(shirt, 3);

         System.out.println("\nTotal Price: " + cart.printBill());

         }

        }
```

OUTPUT :



Program 7: Employee Payable Interface

Problem Statement

Create a class Employee with properties EmpId and Name. Create subclasses HourlyEmployee and SalariedEmployee with different pay rules. Design an interface Payable with method calculatePay(). Implement in subclasses.

CO & BT Mapping

CO2: Analyze a given problem and apply concepts of abstract classes, interfaces, dynamic method dispatch and exception handling

BT Level: Apply
Objectives:

- Applying multiple inheritance using interfaces
- Using super keyword

File Name : Practical07.java

```
interface Payable1 {
        double CalculatePay();
}
class Employeee{
        int EmpId;
        String Name;
        Employeee(int EmpId, String Name){
                this.EmpId=EmpId;
                this.Name=Name;
        }

        void Display() {
                System.out.println("Employee Id = "+ EmpId);
                System.out.println("Employee Name = "+ Name);
        }
}

class HourlyEmployeee extends Employeee implements Payable1{
        double hours;
        double  RatePh;

        HourlyEmployeee(int EmpId, String Name, double hours, double RatePh){
                super (EmpId,Name);
                this.hours=hours;
                this.RatePh=RatePh;
```

```
	}

	public double CalculatePay() {
		return hours * RatePh;
	}
	void Pay() {
		Display();
		System.out.println("Hours = "+ hours);

		System.out.println("Rate Per Hour = " + RatePh);

		System.out.println("Total Salary = " + CalculatePay());
	}
}

class SalariedEmployeee extends Employeee implements Payable1{
	double MonthlySalary;

	SalariedEmployeee(int EmpId, String Name, double MonthlySalary){
		super (EmpId,Name);
		this.MonthlySalary=MonthlySalary;
	}

	public double CalculatePay() {
		double CalculateTotal = MonthlySalary;
		return CalculateTotal;
	}

	void Pay() {
		Display();
		System.out.println("Monthly Salary = "+ MonthlySalary);

		System.out.println("Total Salary = " + CalculatePay());
	}

}

public class Practical7 {

	public static void main(String[] args) {
		// TODO Auto-generated method stub
		System.out.println("Rohit");
		HourlyEmployeee h1= new HourlyEmployeee(101,"Ashutosh",40,1000);
```

```
            SalariedEmployeee s1= new SalariedEmployeee(102,"Rohit", 98000);

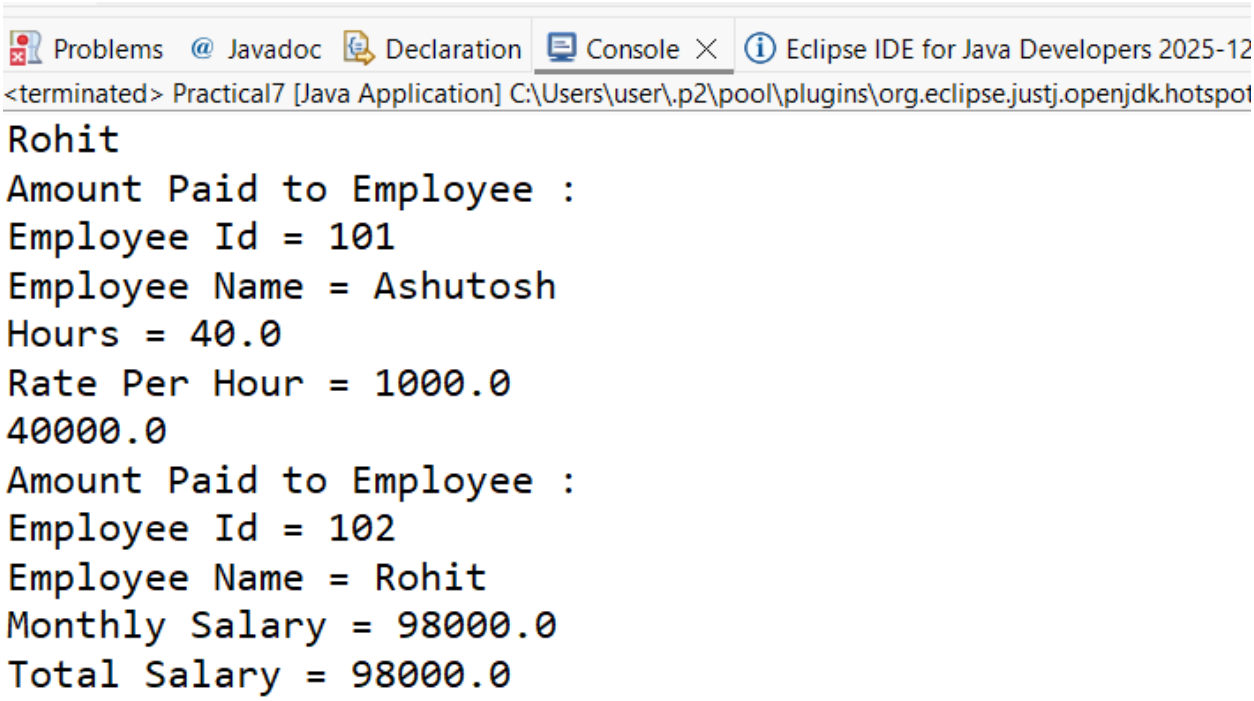

            System.out.println("Amount Paid to Employee : ");
            h1.Pay();

            System.out.println("Amount Paid to Employee : ");
            s1.Pay();

        }

}
```

OUTPUT:



Program 8: Account with Encapsulation and Exception Handling

Problem Statement

Create a class Account with private fields accountNumber and balance. Use constructors, getters, and setters. Create deposit, withdraw, and checkBalance methods. Throw InsufficientFundsException if withdrawal > balance.

CO & BT Mapping

CO2: Analyze a given problem and apply concepts of abstract classes, interfaces, dynamic method dispatch and exception handling

BT Level: Analyze

Objectives:

Analyze a problem statement and apply concepts of Custom/user defined

exceptions Use try, catch, throw keywords

File Name : Practcal08.java

```
class InsufficientFundsException extends Exception {
        public InsufficientFundsException(String message){
                super(message);
        }
}
        class Accounts {
                private String accountNumber;
                private double balance;

                public Accounts(String accountNumber, double balance) {
                   this.accountNumber = accountNumber;
                   this.balance = balance;
                }

                public String getAccountNumber() {
                   return accountNumber;
                }
```

```
    public void setAccountNumber(String accountNumber) {
        this.accountNumber = accountNumber;
    }

    public double getBalance() {
        return balance;
    }

    public void setBalance(double balance) {
        this.balance = balance;
    }

    public void deposit(double amount) {
        balance += amount;
        System.out.println("Amount Deposited: " + amount);
        System.out.println("Updated Balance: " + balance);
    }



    public void withdraw(double amount) throws InsufficientFundsException {
        if (amount > balance) {
            throw new InsufficientFundsException("Insufficient Funds: " + amount);
        }
        balance -= amount;
        System.out.println("Amount Withdrawn: " + amount);
        System.out.println("Remaining Balance: " + balance);
    }



    public void checkBalance() {
        System.out.println("Current Balance: " + balance);
    }
}
public class Practical8 {

    public static void main(String[] args) throws InsufficientFundsException  {
        // TODO Auto-generated method stub
        System.out.println("Rohit");
        Accounts acc = new Accounts("36934311431", 5000.00);

      acc.checkBalance();
```

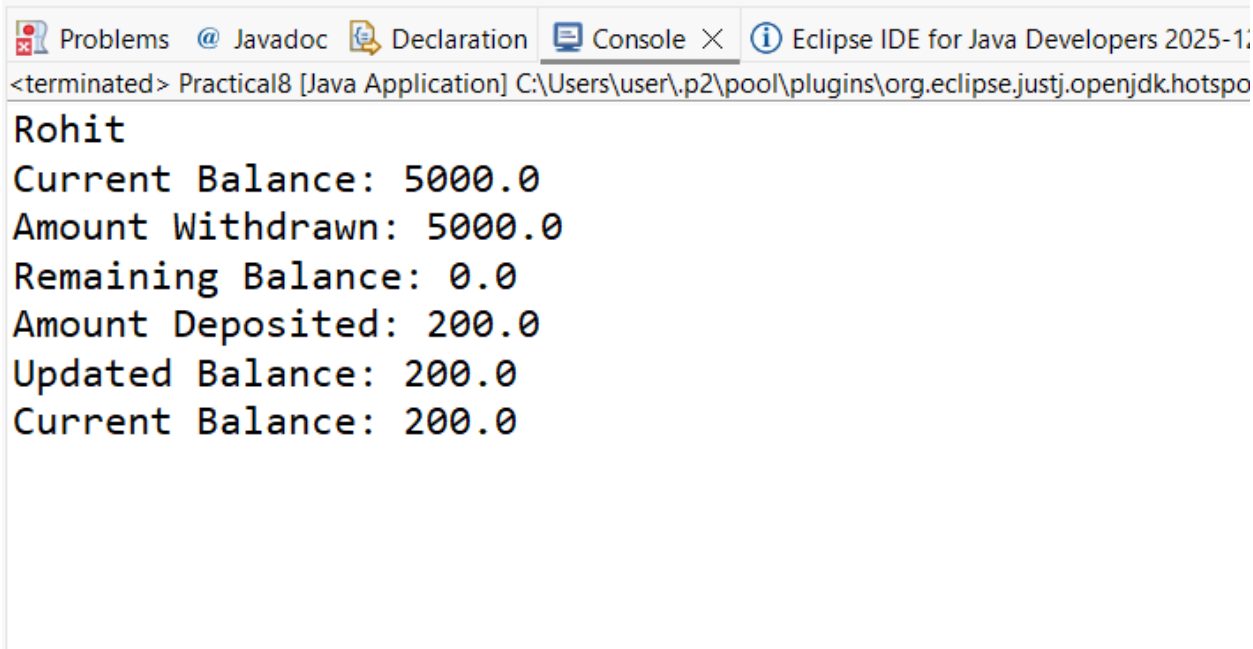
```
            try {
               acc.withdraw(5000);
            } catch (InsufficientFundsException e) {
               System.out.println("Exception: "+ e.getMessage());
            }

            acc.deposit(200);

            acc.checkBalance();
        }

}
```

OUTPUT:



Program 9: Ticket Booking System with Threads

Problem Statement

Simulate a ticket booking system where multiple threads try to book tickets. Use synchronization to avoid overbooking and ensure safe access to available seat count.

CO & BT Mapping

CO3: Analyze a given problem and apply the concepts of multithreading, file handling and Collection frameworks.

BT Level: Analyze Objectives:

Use multithreading

Analyze the problem and use multithreading and apply synchronization where required

File Name: Practical09.java

```
class Ticketbooking{
        int availableSeats;

        Ticketbooking(int availableSeats){
                this.availableSeats = availableSeats;
        }
        synchronized void bookTicket(String CustomerName, int seat) {
                System.out.println(CustomerName + " is trying to book " + seat + " seat");
        if(availableSeats > seat ) {
                System.out.println("Booking successful for " + CustomerName );
                availableSeats -= seat;
                System.out.println("available seats"+ availableSeats);
        } else {
```

```
            System.out.println("Booking failed for " + CustomerName+" available "+seat);
            }
           }
}
class CustomerThread1 extends Thread {
   Ticketbooking booking;
   String customerName;
   int seat;
   public CustomerThread1(Ticketbooking booking, String customerName, int seat) {
      this.booking = booking;
      this.customerName = customerName;
      this.seat = seat;
   }
   @Override
   public void run() {
      booking.bookTicket(customerName, seat);
   }
}
public class Practical09 {
        public static void main(String[] args) throws InterruptedException {
                // TODO Auto-generated method stub
        Ticketbooking booking = new Ticketbooking(5);

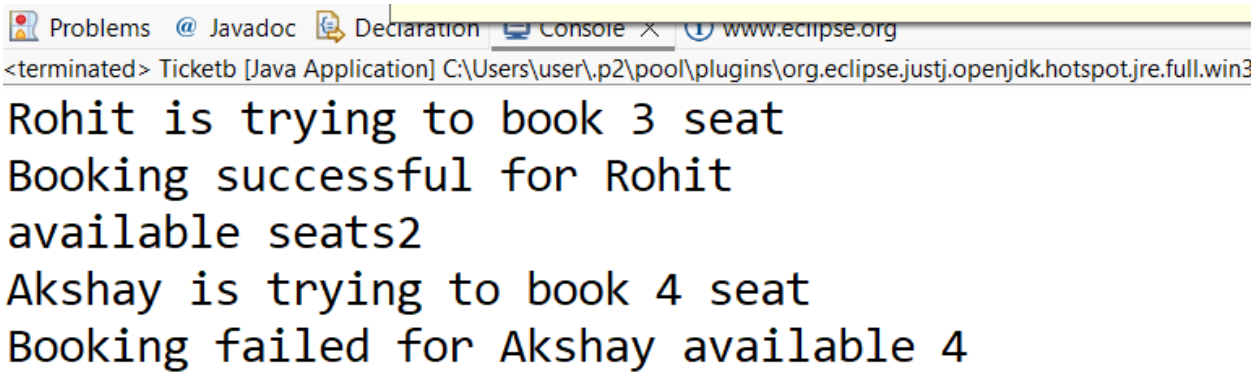
      CustomerThread1 t1 = new CustomerThread1(booking, "Rohit", 3);
      CustomerThread1 t2 = new CustomerThread1(booking, "Akshay", 4);
      CustomerThread1 t3 = new CustomerThread1(booking, "Shubham", 1);
```

```
        CustomerThread1 t4 = new CustomerThread1(booking, "Purva", 1);
        CustomerThread1 t5 = new CustomerThread1(booking, "Vaishnavi", 1);
        CustomerThread1 t6 = new CustomerThread1(booking, "Chaitali", 1);


        t1.start();
        t2.start();
       /* t3.start();
        t4.start();
        t5.start();
        t6.start();*/
        t1.join();
        t2.join();
            }


}
```

OUTPUT :



Program 10: File Reader – Word and Line Count

Problem Statement

Create an application that reads a file and counts the number of words and lines.

CO & BT Mapping

CO3: Analyze a given problem and apply the concepts of multithreading, file handling and Collection frameworks.

BT Level: Apply Objectives:

Understand file handling in Java

File Name: Practical10.java

```
import java.io.FileInputStream;
import java.io.IOException;

public class Practical10{
	public static void main(String[] args) throws IOException {
		// TODO Auto-generated method stub
		System.out.println("Rohit");
		FileInputStream fin = new FileInputStream("d:/File1.txt");

		 int count=0;
		 int line = 0;
		 int c=fin.read();
		while(c != -1) {

			if(c==' ' || c=='\n') {
				count++;
			}

			if ((char)c=='\n') {
				line++;
			}

			c=fin.read();
		}

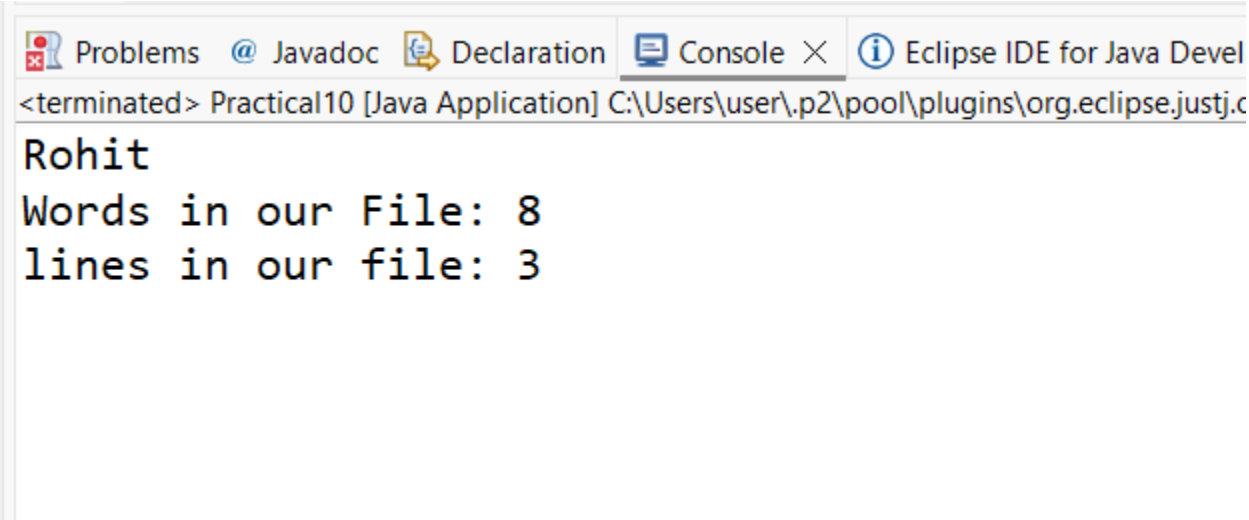
		count++;
		line++;
```

```
            System.out.println("Words in our File: "+count);
            System.out.println("lines in our file: "+line);

        }

}
```

OUTPUT :



Program 11: Remove Duplicates from ArrayList

Problem Statement

Implement a program to remove duplicate elements from an ArrayList of integers and display the list with unique elements only.

CO & BT Mapping

CO3: Analyze a given problem and apply the concepts of multithreading, file handling and Collection frameworks.

BT Level: Apply Objectives:

Analyze and use appropriate collection.

File Name: Practical11.java

```
import java.util.ArrayList;
import java.util.HashSet;

public class Practical11 {

   public static void main(String[] args) {
         System.out.println("Rohit");
         ArrayList<Integer> a = new ArrayList<>();
      a.add(1);
      a.add(2);
      a.add(2);
      a.add(3);
      a.add(4);
      a.add(5);
      a.add(5);

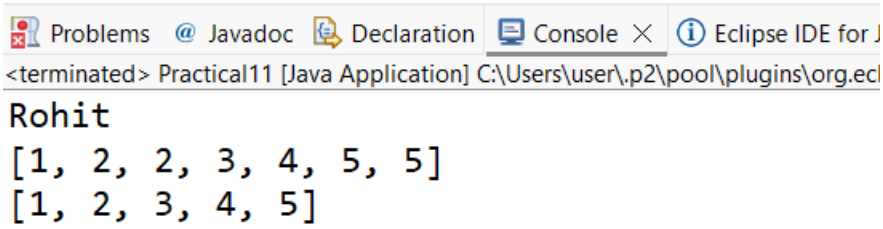
      System.out.println(a);

      HashSet<Integer> hs = new HashSet<>(a);

      System.out.println(hs);

   }
}
```

OUTPUT :


Problems
Javadoc
Declaration
Console
Eclipse IDE for J
<terminated> Practical11 [Java Application] C:\Users\user\.p2\pool\plugins\org.ecl
Rohit
[1, 2, 2, 3, 4, 5, 5]
[1, 2, 3, 4, 5]

Program 12: HashMap with Lambda – Find Topper

Problem Statement

Create a HashMap with name and marks as keys and values. Use lambda to print the topper.

CO & BT Mapping

CO3: Analyze a given problem and apply the concepts of multithreading, file handling and Collection frameworks.

BT Level: Create Objective:

Understand Hashmap

Understand lambda expression

File Name : Practical12.java

```
import java.util.HashMap;
import java.util.Map;

interface Topper {
   void findTopper(HashMap<String, Integer> students);
}



class Practical12 {
   public static void main(String[] args) {
      System.out.println("Rohit");
      HashMap<String, Integer> students = new HashMap<>();

      students.put("Bhushan", 78);
      students.put("Chandan", 95);
      students.put("Rahul", 70);
      students.put("Aditya", 98);
      students.put("Mukesh", 65);
      students.put("Lakhan", 87);




      Topper t = (s) -> {
         String topperName = "";
```

```
        int maxMarks = 0;


        for (HashMap.Entry<String, Integer> entry : s.entrySet()) {

            if (entry.getValue() > maxMarks) {
                maxMarks = entry.getValue();
                topperName = entry.getKey();
            }
        }

        System.out.println("Topper: " + topperName + " with marks: " + maxMarks);
    };




    t.findTopper(students);
  }
}
```

OUTPUT :

